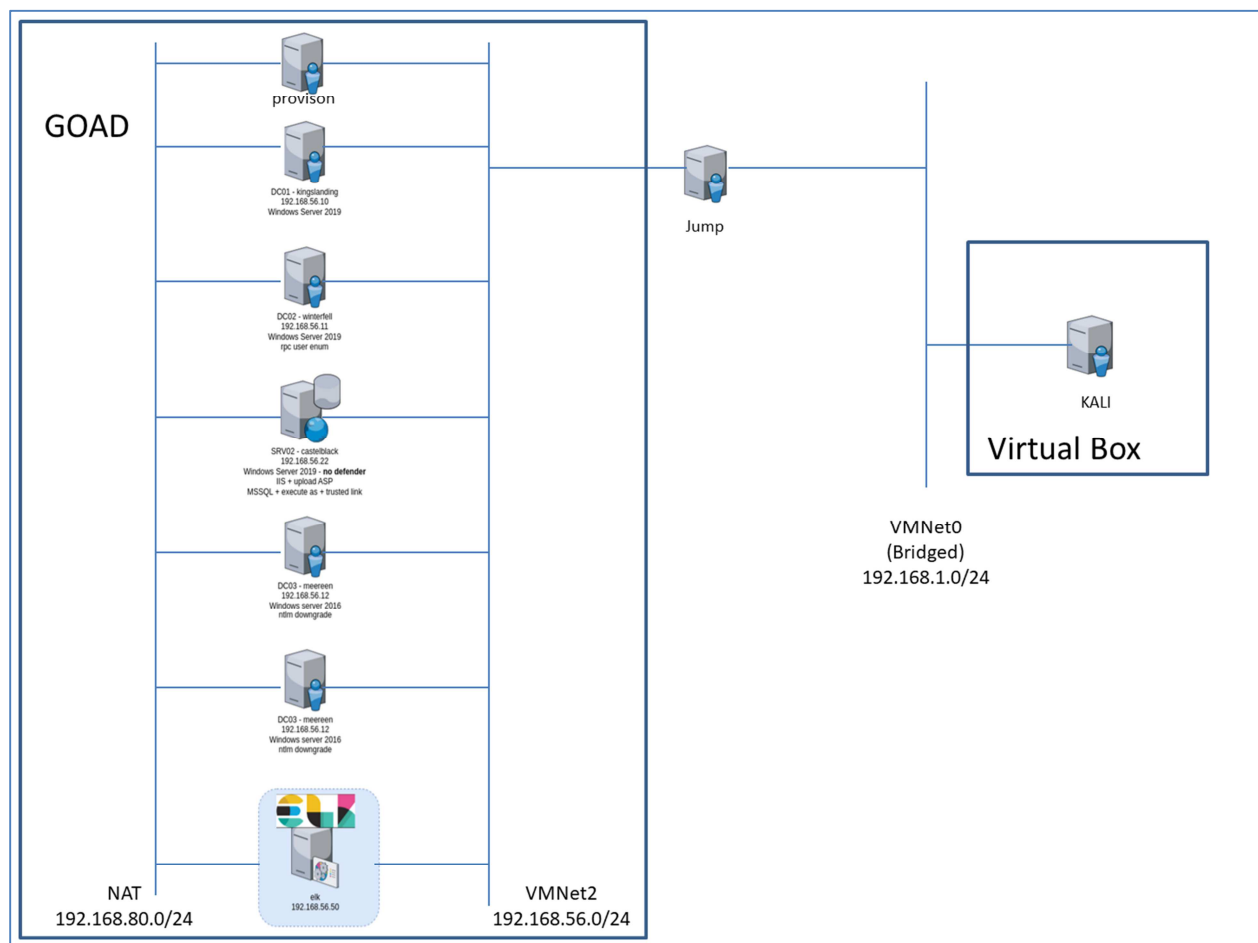


GOAD Lab Access via a Jump Box

JJKirn 1/6/26

Intro:

This document shows a method to add a “**Jump**” box VM to an existing operational **GOAD LAB** running on a **VMware Workstation Pro** instance on a **Windows 11 host** to provide remote access. The remote access can even be via a **Kali VM** running on **Virtual Box** on a **separate Windows host** system as long as there is a **shared bridged network**.

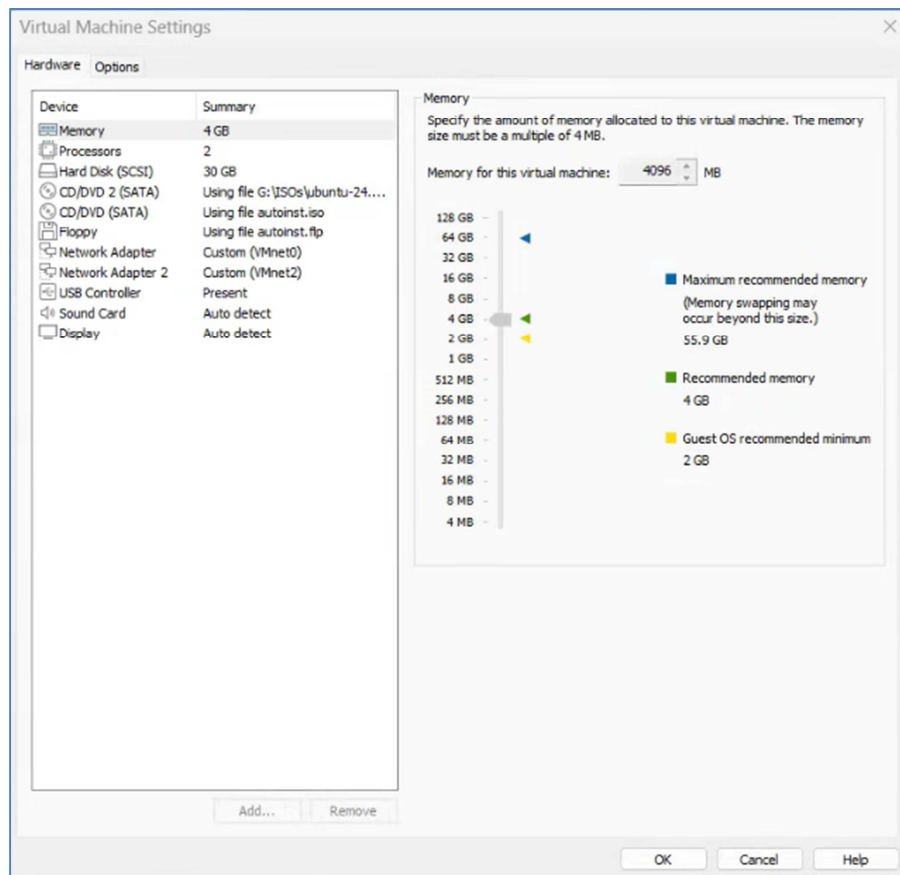


GOAD LAB Network with Jump Box and Remote Windows host Virtualbox Kali VM

Start with the “Jump” box VM:

Step 1: Create the “Jump” box VM on the same VMware Workstation that is running the GOAD lab.

I chose **Ubuntu 24.04 Desktop** for the VM. Here is the configuration I used:



Once you power it up, you will need to answer the usual questions about username and password, hostname, timezone, etc... to be able to create the VM.

First step is to do the usual update/upgrade. Open a terminal window and enter:

```
$ sudo apt upgrade  
$ sudo apt upgrade
```

Step 2: Configure the “Jump” box for static IP

As root user, edit `/etc/netplan/50-cloud-init.yml` file as follows:

```
root@openvpn: /etc/netplan
jim@openvpn:/etc/netplan$
jim@openvpn:/etc/netplan$ ls
50-cloud-init.yaml
jim@openvpn:/etc/netplan$ cat 50-cloud-init.yaml
cat: 50-cloud-init.yaml: Permission denied
jim@openvpn:/etc/netplan$ sudo su
[sudo] password for jim:
root@openvpn:/etc/netplan# cat 50-cloud-init.yaml
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.1.110/24
      nameservers:
        addresses: [8.8.8.8,8.8.4.4]
      routes:
        - to: default
          via: 192.168.1.1
root@openvpn:/etc/netplan#
```

As you can see, I chose **192.168.1.110** as the **static IP** for the “Jump” box. To make these changes effective run:

```
$ sudo netplan apply
```

Verify ip address is active by running the **ip a** command:

```
jim@openvpn:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:23:b3:99 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.110/24 brd 192.168.1.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe23:b399/64 scope link
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:23:b3:a3 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 192.168.56.130/24 brd 192.168.56.255 scope global dynamic noprefixroute ens37
        valid_lft 1196sec preferred_lft 1196sec
    inet6 fe80::9eb1:455e:6637:8000/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Step 3: Make sure ssh server is running:

Run the following command to install openssh server:

```
$ sudo apt install openssh-server
```

Next Steps are on the remote Windows host Kali VM

Step 1: Start by installing a Kali VM (using VirtualBox in my case)

I assume you know how to download a **Kali ISO** and install it.

Remember we are doing these steps on a **remote Windows host machine** using **VirtualBox**.

Kali settings:

```
Name: kali
Base Memory: 4GB
Displays: 1
Storage: 120GB
Network Adapter 1: Bridged
```

Step 2: Set a static IP

I chose to assign a **static IP to the VM (192.168.1.200)**.

As root user edit /etc/network/interfaces file:

```
(root@kali)-[/etc/network]
# cat interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.1.200
    netmask 255.255.255.0
    gateway 192.168.1.1
    #dns-nameserver 8.8.8.8
```

Restart networking to make changes effective:

```
(jim@kali)-[~/Desktop]
$ sudo systemctl restart networking
```

Verify Changes worked by running ip a:

```
(jim@kali)-[~/Desktop]
$ ip a
```

```

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state
UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8b:7c:42 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.200/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe8b:7c42/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

```

Step 3: Make sure ssh server is running:

Run the following command to install openssh server:

```
$ sudo apt install openssh-server
```

Step 4: Create a SSH config file at ~/.ssh/config:

```

(jim@kali)-[~/.ssh]
└─$ cat config
Host jumphost
    HostName 192.168.1.110
    User jim

Host 192.168.56.*
    ProxyCommand ssh -W %h:%p jumphost
    User vagrant

```

Step 5: Configure proxychains4

```

# Install proxychains
sudo apt install proxychains4

# Edit /etc/proxychains4.conf:

# Add this at end of file
socks5 127.0.0.1 9050

```

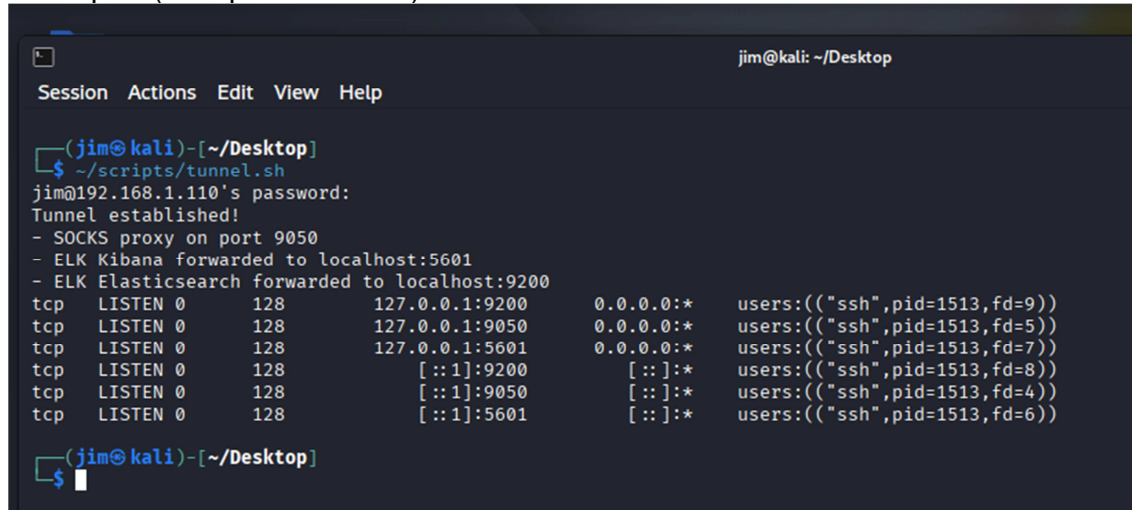
You are now ready to access the **GOAD Lab** through the use of **proxychains4**. Below are some example scripts that will help simply use of various tools.

Script Examples:

1. Create script for creating the ssh tunnel (tunnel.sh):

```
(jim@kali)-[~/scripts]
└─$ cat tunnel.sh
#!/bin/bash
ssh -D 9050 -L 5601:192.168.56.50:5601 -L
9200:192.168.56.50:9200 -f -N -o ServerAliveInterval=60 -o
ServerAliveCountMax=3 jim@192.168.1.110
echo "Tunnel established!"
echo "- SOCKS proxy on port 9050"
echo "- ELK Kibana forwarded to localhost:5601"
echo "- ELK Elasticsearch forwarded
to localhost:9200"
ss -tulnp | grep -E "9050|5601|9200"
```

Example: (~/scripts/tunnel.sh)



```
jim@kali: ~/Desktop
Session Actions Edit View Help

(jim@kali)-[~/Desktop]
└─$ ~/scripts/tunnel.sh
jim@192.168.1.110's password:
Tunnel established!
- SOCKS proxy on port 9050
- ELK Kibana forwarded to localhost:5601
- ELK Elasticsearch forwarded to localhost:9200
tcp LISTEN 0 128 127.0.0.1:9200 0.0.0.0:* users:(("ssh",pid=1513,fd=9))
tcp LISTEN 0 128 127.0.0.1:9050 0.0.0.0:* users:(("ssh",pid=1513,fd=5))
tcp LISTEN 0 128 127.0.0.1:5601 0.0.0.0:* users:(("ssh",pid=1513,fd=7))
tcp LISTEN 0 128 [::]:9200 [::]:* users:(("ssh",pid=1513,fd=8))
tcp LISTEN 0 128 [::]:9050 [::]:* users:(("ssh",pid=1513,fd=4))
tcp LISTEN 0 128 [::]:5601 [::]:* users:(("ssh",pid=1513,fd=6))

(jim@kali)-[~/Desktop]
└─$
```

Note: The above script must be run at least once before running any of the following scripts or any proxychain4 commands.

For the following scripts you need to add one of the five server names (kingslanding, castelblack, ...) for them to run correctly.

You can verify that the tunnel is operational at any time by running `ss -tulnp`:

```

jim@kali: ~/Desktop
Session Actions Edit View Help

(jim@kali)-[~/Desktop]
$ ss -tunlp
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port      Process
tcp        LISTEN     0           128         127.0.0.1:9200          0.0.0.0:*               users:((("ssh",pid=1513,fd=9))
tcp        LISTEN     0           128         127.0.0.1:9050          0.0.0.0:*               users:((("ssh",pid=1513,fd=5))
tcp        LISTEN     0           128         127.0.0.1:5601          0.0.0.0:*               users:((("ssh",pid=1513,fd=7))
tcp        LISTEN     0           128         [::]:9200               [::]:*                  users:((("ssh",pid=1513,fd=8))
tcp        LISTEN     0           128         [::]:9050               [::]:*                  users:((("ssh",pid=1513,fd=4))
tcp        LISTEN     0           128         [::]:5601               [::]:*                  users:((("ssh",pid=1513,fd=6))

(jim@kali)-[~/Desktop]
$

```

2. This script launches a xfreerdp session to the selected server:

```

(jim@kali)-[~/scripts]
$ cat xfreerdp.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 xfreerdp3 /compression /cert:ignore +auto-reconnect
/v:$IP /u:vagrant /p:vagrant +clipboard

```

Example: (~/scripts/xfreerdp.sh kingslanding)

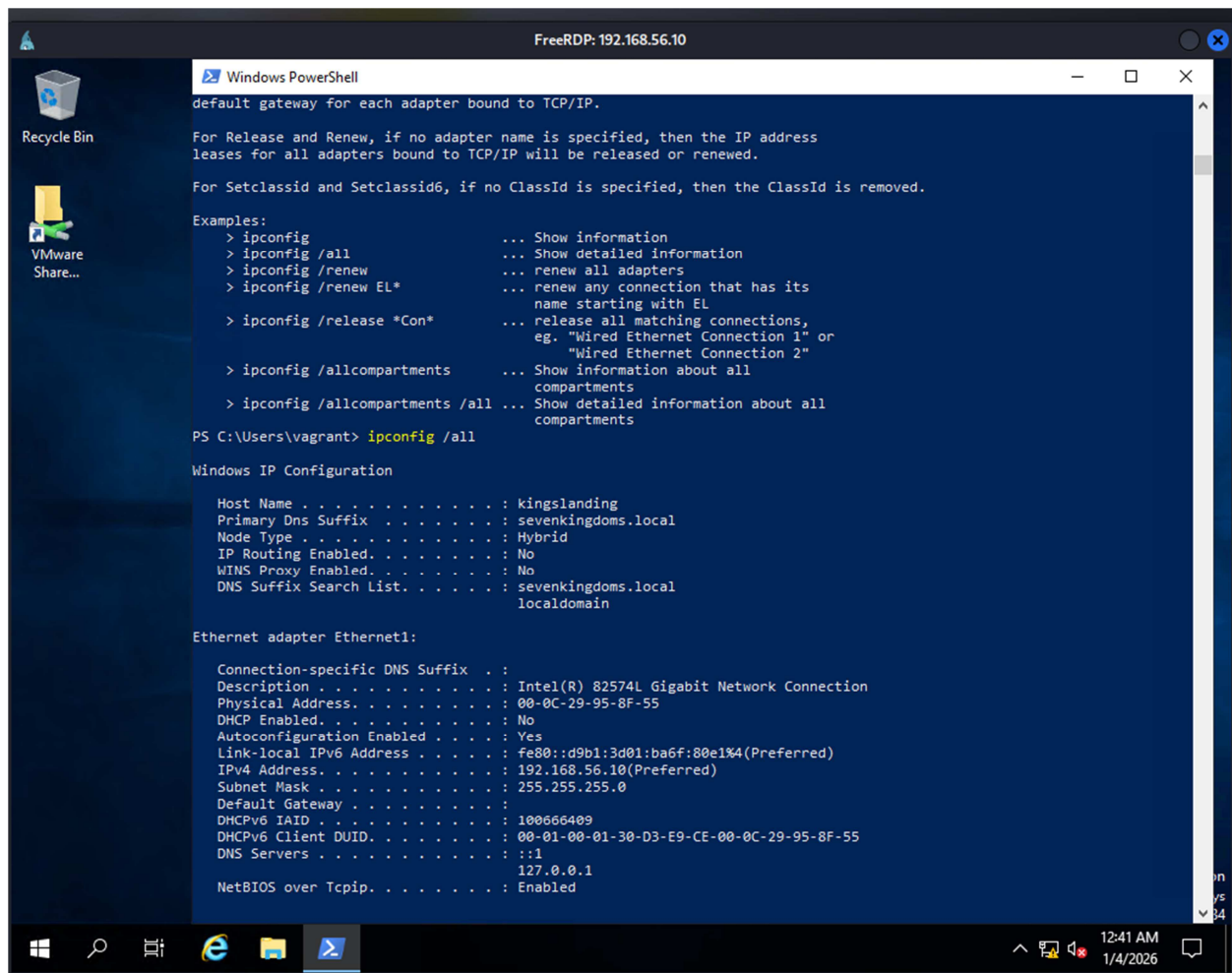
```

jim@kali: ~/Desktop/scripts
Session Actions Edit View Help

(jim@kali)-[~/Desktop/scripts]
$ ./xfreerdp.sh kingslanding
Connecting to kingslanding (192.168.56.10) ...
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[02:40:49:963] [3733:00000e95] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Using /p is insecure
[02:40:49:963] [3733:00000e95] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Passing credentials or secrets via command line might expose these in the process list
[02:40:49:963] [3733:00000e95] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Consider using one of the following (more secure) alternatives:
[02:40:49:963] [3733:00000e95] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: - /args-from: pipe in arguments from stdin, file or file descriptor
[02:40:49:963] [3733:00000e95] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: - /from-stdin pass the credential via stdin
[02:40:49:963] [3733:00000e95] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: - set environment variable FREERDP_ASKPASS to have a gui tool query for credentials
[02:40:49:969] [3733:00000e98] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]: : keycode: 0x08 → no RDP scancode found
[02:40:49:969] [3733:00000e98] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]: ZEHA: keycode: 0x5d → no RDP scancode found
[proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.10:3389 ... OK
[02:40:50:038] [3733:00000e98] [WARN][com.freerdp.crypto] - [tls_verify_certificate]: [DANGER] Certificate not checked, /cert:ignore in use.
[02:40:50:038] [3733:00000e98] [WARN][com.freerdp.crypto] - [tls_verify_certificate]: [DANGER] This prevents MITM attacks from being detected!
[02:40:50:038] [3733:00000e98] [WARN][com.freerdp.crypto] - [tls_verify_certificate]: [DANGER] Avoid using this unless in a secure LAN (no internet) environment
[02:40:50:039] [3733:00000e98] [ERROR][com.winpr.sspi.Kerberos] - [kerberos.AcquireCredentialsHandleA]: krb5_parse_name (Configuration file does not specify default realm [-1765328160])
[02:40:50:039] [3733:00000e98] [ERROR][com.winpr.sspi.Kerberos] - [kerberos.AcquireCredentialsHandleA]: krb5_parse_name (Configuration file does not specify default realm [-1765328160])
[02:40:50:264] [3733:00000e98] [INFO][com.freerdp.gdi] - [gdi_init_ex]: Local framebuffer format PIXEL_FORMAT_BGRX32
[02:40:50:264] [3733:00000e98] [INFO][com.freerdp.gdi] - [gdi_init_ex]: Remote framebuffer format PIXEL_FORMAT_BGRX32
[02:40:50:283] [3733:00000e98] [INFO][com.freerdp.channels.rdpnd.client] - [rdpsnd_load_device_plugin]: [static] Loaded fake backend for rdpnd
[02:40:50:283] [3733:00000e98] [INFO][com.freerdp.channels.drdynvc.client] - [dvcman_load_addin]: Loading Dynamic Virtual Channel ainput
[02:40:50:283] [3733:00000e98] [INFO][com.freerdp.channels.drdynvc.client] - [dvcman_load_addin]: Loading Dynamic Virtual Channel rdpgfx
[02:40:50:283] [3733:00000e98] [INFO][com.freerdp.channels.drdynvc.client] - [dvcman_load_addin]: Loading Dynamic Virtual Channel disp
[02:40:50:283] [3733:00000e98] [INFO][com.freerdp.channels.drdynvc.client] - [dvcman_load_addin]: Loading Dynamic Virtual Channel rdpnd
[02:40:50:611] [3733:00000e9f] [INFO][com.freerdp.channels.rdpnd.client] - [rdpsnd_load_device_plugin]: [dynamic] Loaded fake backend for rdpnd
[02:40:51:400] [3733:00000e9f] [INFO][com.freerdp.channels.rdpnd.client] - [rdpsnd_load_device_plugin]: [dynamic] Loaded fake backend for rdpnd

```

And the xfreerdp window appears as:



3. This script launches an evil-winrm session to the selected server:

```
(jim@ kali)-[~/scripts]
$ cat winrm-connect.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 evil-winrm -i $IP -u vagrant -p vagrant
```

Example: (~/scripts/winrm-connect.sh castelblack)


```

(jim@kali)-[~/Desktop/scripts]
└─$ ./winrm-connect.sh castelblack
Connecting to castelblack (192.168.56.22)...
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17

Evil-WinRM shell v3.9

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
[proxychains] Strict chain  ...  127.0.0.1:8080  ...  192.168.56.22:5985  ...  OK
*Evil-WinRM* PS C:\Users\vagrant\Documents>
*Evil-WinRM* PS C:\Users\vagrant\Documents> ls

Directory: C:\Users\vagrant\Documents

Mode                LastWriteTime         Length Name
----                -
d-----         12/17/2025   12:37 PM             Visual Studio 2017
d-----         12/17/2025   10:21 AM             WindowsPowerShell

*Evil-WinRM* PS C:\Users\vagrant\Documents>

```

4. This script launches a PSEXEC session to the selected server:

```

(jim@kali)-[~/scripts]
└─$ cat psexec.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 impacket-psexec vagrant:vagrant@$IP

```

Example: (~/scripts/psexec-connect.sh meereen)

```
jim@kali: ~/Desktop/scripts
Session Actions Edit View Help

(jim@kali)-[~/Desktop/scripts]
└─$ ./psexec-connect.sh meereen
Connecting to meereen (192.168.56.12)...
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.12:445 ... OK
[*] Requesting shares on 192.168.56.12.....
[*] Found writable share ADMIN$
[*] Uploading file aHmnMCrf.exe
[*] Opening SVCManager on 192.168.56.12.....
[*] Creating service szSd on 192.168.56.12.....
[*] Starting service szSd.....
[proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.12:445 ... OK
[proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.12:445 ... OK
[!] Press help for extra shell commands
[proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.12:445 ... OK
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32> powershell
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Windows\system32>
pwd
PS C:\Windows\system32> pwd

Path
____
C:\Windows\system32
```

5. This script launches a nxc smb session to the selected server:

```
(jim@kali)-[~/scripts]
└─$ cat nxc-smb.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 nxc smb -u vagrant -p vagrant -x "type
C:\Windows\System32\drivers\etc\hosts"
```

Example: (~/scripts/nxc-smb.sh meereen)

```

jim@kali: ~/Desktop/scripts
Session Actions Edit View Help
(jim@kali)~-[~/Desktop/scripts]
$ ./nxc-smb.sh essos
Connecting to essos (192.168.56.12
192.168.56.23) ...
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
Running nxc against 2 targets
proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.12:445 ... 192.168.56.23:445 ... OK
... OK
[proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.12:135 [proxychains] Strict chain ... 127.0.0.1:8080 ... 192.16
8.56.23:135 ... OK
... OK
Running nxc against 2 targets
... 192.168.56.12:445 [proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.23:445 ... OK
... OK
SMB 192.168.56.12 445 MEEREEN [+] Windows 10 / Server 2016 Build 14393 x64 (name:MEEREEN) (domain:essos.local)
(signing:True) (SMBv1:True)
Running nxc against 2 targets
SMB 192.168.56.23 445 BRAAVOS [+] Windows 10 / Server 2016 Build 14393 x64 (name:BRAAVOS) (domain:essos.local)
(signing:False) (SMBv1:True)
Running nxc against 2 targets
SMB 192.168.56.12 445 MEEREEN [+] essos.local\vagrant:vagrant (Pwn3d!)
Running nxc against 2 targets
proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.23:445 ... 192.168.56.12:135 ... OK
... OK
Running nxc against 2 targets
... 192.168.56.12:49666 ... OK
[proxychains] Strict chain ... 127.0.0.1:8080 ... 192.168.56.12:49666 ... OK
SMB 192.168.56.23 445 BRAAVOS [+] essos.local\vagrant:vagrant
SMB 192.168.56.12 445 MEEREEN [+] Executed command via wmiexec
SMB 192.168.56.12 445 MEEREEN # Copyright (c) 1993-2009 Microsoft Corp.
SMB 192.168.56.12 445 MEEREEN # This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
SMB 192.168.56.12 445 MEEREEN #
SMB 192.168.56.12 445 MEEREEN # This file contains the mappings of IP addresses to host names. Each
SMB 192.168.56.12 445 MEEREEN # entry should be kept on an individual line. The IP address should
SMB 192.168.56.12 445 MEEREEN # be placed in the first column followed by the corresponding host name.
SMB 192.168.56.12 445 MEEREEN # The IP address and the host name should be separated by at least one
SMB 192.168.56.12 445 MEEREEN # space.
SMB 192.168.56.12 445 MEEREEN #
SMB 192.168.56.12 445 MEEREEN # Additionally, comments (such as these) may be inserted on individual
SMB 192.168.56.12 445 MEEREEN # lines or following the machine name denoted by a '#' symbol.
SMB 192.168.56.12 445 MEEREEN #
SMB 192.168.56.12 445 MEEREEN # For example:
SMB 192.168.56.12 445 MEEREEN #
SMB 192.168.56.12 445 MEEREEN # 102.54.94.97 rhino.acme.com # source server
SMB 192.168.56.12 445 MEEREEN # 38.25.63.10 x.acme.com # x client host
SMB 192.168.56.12 445 MEEREEN # localhost name resolution is handled within DNS itself.
SMB 192.168.56.12 445 MEEREEN # 127.0.0.1 localhost
SMB 192.168.56.12 445 MEEREEN # ::1 localhost
Running nxc against 2 targets 100% 0:00:00
```

The above scripts are just some examples of how you can automate using proxychains4 to perform usual pentesting commands on a remote system through the use of a **Jump box VM**.

Obviously you can use **proxychains4** directly with any typical Kali attack tool.

Enjoy!